Trip Genius AI — Main Project Report

Trip Genius AI: An AI-Powered Travel Planner

Abstract

Trip Genius AI is a browser-based travel-planning application that creates a personalized itinerary draft from a traveller's starting city, destination, dates, budget, number of travellers, interests and travel style. The backend requests structured output from a hosted generative AI model, validates the response, and saves trip and itinerary records in Supabase. The frontend supports viewing saved trips, reopening and deleting a trip, printing or saving an itinerary, and searching for hotels and restaurants through Google Maps query links.

The project follows a clear separation of concerns. A React and TypeScript frontend, deployed on Vercel, is responsible only for collecting input and presenting results. A Python FastAPI backend, deployed on Render, owns validation, AI orchestration and persistence. This split keeps secret keys on the server, makes each layer easy to test on its own, and allows the interface or the AI provider to be changed without rewriting the rest of the system.

The application is intentionally positioned as a planning assistant. It does not book transport or accommodation, and it does not guarantee prices or availability. Its value is in removing the blank-page problem: a traveller receives a structured, day-by-day starting point that can be reviewed, corrected and saved.

Project at a glance

Item	Details
Project name	Trip Genius AI
Frontend	React, TypeScript and Vite, hosted on Vercel
Backend	Python and FastAPI, hosted on Render
AI service	Groq SDK with the openai/gpt-oss-20b model
Database	Supabase (trip and itinerary records)
Key features	Itinerary generation, saved trips, hotel and restaurant search links, print / save as PDF

How this report is organised

Section	Content	Page
1-2	Project description, objectives, scope and usage scenarios	2-3
3-4	Technical architecture, prerequisites and configuration	4-5
5-6	Project workflow and backend API overview	6
7	Website pages with screenshots	7-8
8-10	Testing plan, limitations, security, future scope, deployment and conclusion	9-10

1. Project Description

The purpose of Trip Genius AI is to simplify the first stage of trip planning. Instead of starting with a blank page, a traveller can submit trip preferences and receive a structured day-wise itinerary to review and adjust. The application is designed as an assistant for planning, not as a booking or travel-provider platform.

Problem addressed

Planning commonly involves coordinating destination, dates, group size, budget, activities and food preferences across multiple sources. A traveller typically opens several websites, copies details between notes and spreadsheets, and repeatedly rebalances the plan when one constraint changes. The project combines these inputs in one interface and produces an itinerary draft that can be saved and revisited.

Project objectives

- **Collect essential trip preferences** in a simple form with only the fields needed to shape a plan.
- **Generate a day-by-day plan** with activity periods, estimated costs and travel tips.
- **Persist trip details and itinerary output** in Supabase so work is not lost when the page is closed.
- **Allow users to list, reopen and delete** saved trips from a single Saved Trips collection.
- **Provide destination-based Google Maps searches** for hotels and restaurants, including food-preference query terms.
- **Support browser printing or Save as PDF** for the itinerary so it can be taken offline.

Target users

The system is aimed at independent travellers, students, small groups and families who want a quick, personalised starting plan rather than a fully managed tour. Because the inputs are limited to common planning details, the application is approachable for first-time users and does not require an account in its current version.

Scope of the project

In scope	Out of scope (current version)
Preference form and input validation	Booking of flights, trains, hotels or activities
AI-generated, structured itinerary with estimated costs	Guaranteed or live prices and availability
Saving, reopening and deleting trips in Supabase	User accounts and per-user trip ownership
Google Maps search links for hotels and restaurants	Ratings, reviews and opening hours retrieved by the app
Print / Save as PDF through the browser	Server-side PDF generation

Design principles

- **Simplicity first:** one form, one itinerary view and one saved-trips list.
- **Structured output:** the AI response is checked against an expected schema before it is stored or shown.
- **Transparency:** costs are labelled as estimates and external searches are clearly external.
- **Security by separation:** API keys and database credentials stay on the backend.

2. Project Description and Scenarios

The following scenarios describe how a traveller is expected to use the application from first visit to final output. Each scenario maps to one or more backend endpoints described later in this report.

Scenario 1: Generate a personalized itinerary

A traveller enters Bhopal as the starting city and Kerala as the destination, chooses travel dates, sets a 20,000 INR budget for two travellers, selects Nature, Sightseeing and Food as interests, and chooses Adventure as the travel style. The app submits the details to the backend and displays a generated itinerary with day-wise activities and estimated costs. The figures are planning estimates and should be checked before travel.

Outcome: a structured itinerary containing a destination summary, travel dates, activities grouped by day and time period, an estimated cost overview and travel tips, which is also saved automatically for later use.

Scenario 2: Save and revisit a trip

After reviewing the generated itinerary, the traveller can use Saved Trips to return to previously saved travel plans. The backend stores trip details and itinerary data in Supabase. A saved itinerary can be opened for review or deleted through the available controls.

Outcome: a persistent list of earlier plans (for example, a Pachmarhi trip stored from an earlier session) that can be reopened without calling the AI model again.

Scenario 3: Find hotels and restaurants

The Hotels & Restaurants section constructs Google Maps search URLs for a destination. Restaurant searches can include all, vegetarian, non-vegetarian or Jain preferences. These links lead to external search results; the app does not directly retrieve or verify each place’s ratings, prices, opening hours, availability or food preparation.

Outcome: one click from the itinerary to a Google Maps search that is already specific to the destination and food preference.

Scenario 4: Print or save the plan

The user can open the browser print dialogue from the itinerary view and select a printer or Save as PDF. The exact print layout depends on the browser and its print settings.

Outcome: a portable copy of the itinerary that can be shared with co-travellers or kept offline.

Scenario summary

Scenario	Trigger	Main component involved	Result
1. Generate itinerary	Form submitted	Frontend, FastAPI, Groq, Supabase	Itinerary shown and saved
2. Save and revisit	Open Saved Trips	Frontend, FastAPI, Supabase	Stored trip reopened or deleted
3. Hotels and restaurants	Search action chosen	FastAPI, Google Maps URLs	External search opened
4. Print or save	Print action chosen	Browser print dialogue	Printed copy or PDF

3. Technical Architecture

The application uses a separate frontend and backend. The React/TypeScript frontend runs on Vercel and sends JSON requests to the FastAPI backend hosted on Render. The backend calls Groq for itinerary generation and Supabase for persistence. Hotel and restaurant endpoints return Google Maps search URLs.

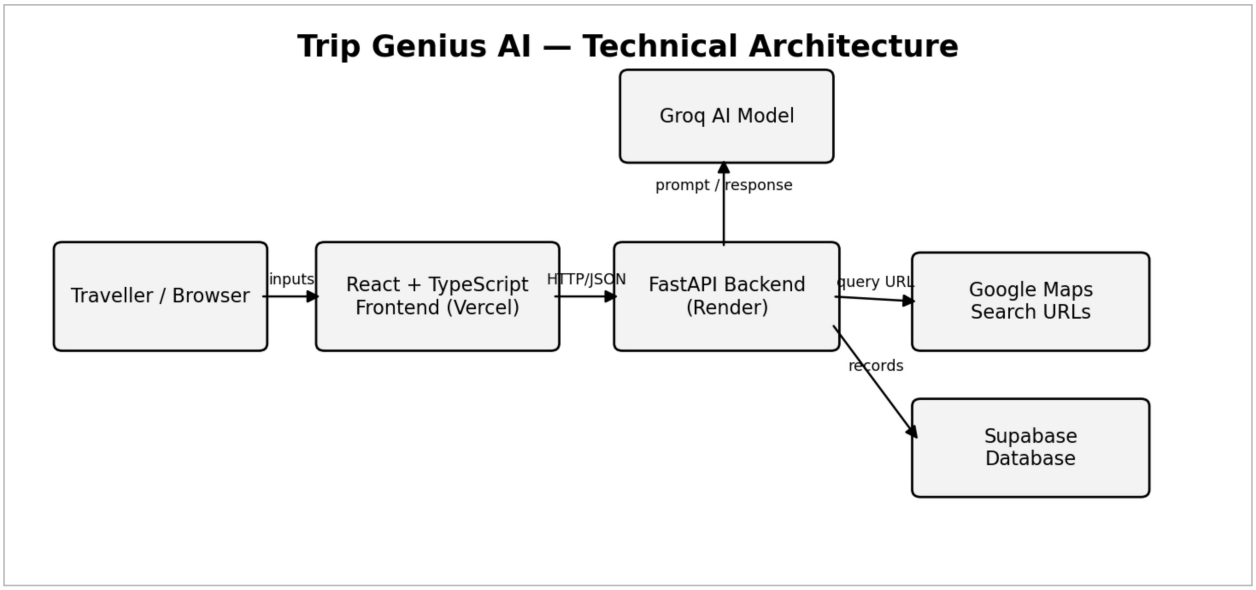


Figure 1. High-level architecture of Trip Genius AI.

How the layers interact

The traveller interacts only with the browser. The frontend converts form values into a JSON request and calls the backend over HTTP. The backend validates the request, builds a prompt, calls the Groq model, checks the structure of the reply and writes the records to Supabase before returning the itinerary to the frontend. Hotel and restaurant requests do not involve the AI model; the backend simply builds a destination-specific Google Maps query URL.

Component	Technology	Responsibility
Frontend	React, TypeScript, Vite	Collect inputs and display itinerary, saved trips and search controls
Backend	Python, FastAPI	API routing, validation, AI orchestration and persistence
AI service	Groq SDK; openai/gpt-oss-20b	Generate structured itinerary content
Validation	Pydantic and response checks	Check request and itinerary structure
Database	Supabase	Store trip and itinerary records
Hosting	Vercel and Render	Deploy frontend and backend separately
External search	Google Maps query URLs	Open destination-specific place searches

Why this architecture

- **Security:** the Groq key and database credentials never reach the browser.
- **Independent deployment:** the frontend and backend can be redeployed separately.
- **Replaceable parts:** the AI provider or the database can be changed by editing one backend module.

4. Prerequisites and Configuration

The following items are needed to run the project locally and to deploy it. Each is listed with the reason it is required.

Prerequisite	Purpose
Python 3 and pip	Create the backend environment and install FastAPI, Pydantic, the Groq SDK and the Supabase client
Node.js and npm	Install dependencies, run and build the React/Vite frontend
Groq API key (private backend variable)	Authenticate calls to the AI model for itinerary generation
Supabase project and database configuration	Store saved trips and itinerary records
Git and a modern web browser with internet access	Clone the repository, run the app and view results
Vercel and Render project settings	Deploy the frontend and backend respectively

Configuration values

Secrets such as GROQ_API_KEY and Supabase credentials must remain in environment configuration and must not be pasted into documentation, screenshots or public source control. The supplied source archive does not include a production .env file, which is appropriate for protecting credentials.

Setting	Where it is set	Notes
GROQ_API_KEY	Backend environment (Render)	Server-side only; never exposed to the frontend
Supabase URL and key	Backend environment (Render)	Used by the database client in the backend
Backend base URL	Frontend configuration (Vercel)	Points the frontend to the deployed API
Allowed origins (CORS)	Backend configuration	Should list only the deployed frontend origin

Local setup outline

1. Clone the repository and open the project folder.
2. In the backend folder, create a virtual environment, install the requirements with pip and provide the private environment variables.
3. Start the FastAPI server and confirm that the /health endpoint responds.
4. In the frontend folder, run npm install, set the backend URL and start the Vite development server.
5. Open the local address in a browser and generate a test itinerary.

Deployment checklist

- Backend deployed on Render with all private variables entered in its dashboard.
- Frontend deployed on Vercel with the backend URL configured.
- CORS restricted to the Vercel domain.
- The /health endpoint checked after every deployment.

5. Project Workflow

The end-to-end flow from a submitted form to a stored and displayed itinerary is shown below, with the purpose of each step.

Step	Action	Purpose
1	The user enters starting city, destination, dates, budget, traveller count, interests and travel style in the frontend.	Capture the preferences that shape the plan
2	The frontend sends the trip request as JSON to the FastAPI itinerary endpoint.	Hand the request to the backend
3	Pydantic models validate the incoming request.	Reject missing or malformed data early
4	The AI service builds a prompt and calls the Groq model.	Obtain a draft itinerary
5	The backend parses the model response and checks the expected itinerary fields and day/activity structure.	Ensure the output is usable and complete
6	The backend saves trip details and itinerary output in Supabase.	Make the plan persistent
7	The frontend displays the itinerary and allows the user to save, reopen, delete or print.	Present and manage the result
8	Hotel and restaurant actions request destination-based Google Maps search URLs.	Help the user find places to stay and eat

6. Backend API Overview

The backend exposes a small REST interface. All request and response bodies are JSON.

Method	Endpoint	Purpose
GET	/health	Health-check response used to confirm that the backend is running
POST	/api/itinerary	Generate itinerary and save records
GET	/api/trips	List saved trips
GET	/api/trips/{trip_id}	Retrieve one saved trip
DELETE	/api/trips/{trip_id}	Delete trip and associated itinerary
GET	/api/places/hotels	Return hotel Google Maps search URL
GET	/api/places/restaurants	Return restaurant search URL with food preference

Notes on the API

- Only POST /api/itinerary calls the AI model; every other endpoint is a read, a delete or a URL builder.
- Saving happens as part of itinerary generation, so the Saved Trips list is always in step with generated plans.

7. Exploring the Website’s Web Pages

7.1 Trip Planner form

The form collects the minimum details required to shape an itinerary. The supplied screenshot shows a Bhopal-to-Kerala example, dates from 21 to 23 October 2026, a 20,000 INR budget, two travellers, Nature/Sightseeing/Food interests and Adventure style. Interests are entered as comma-separated text, and the travel style is chosen from a drop-down list.

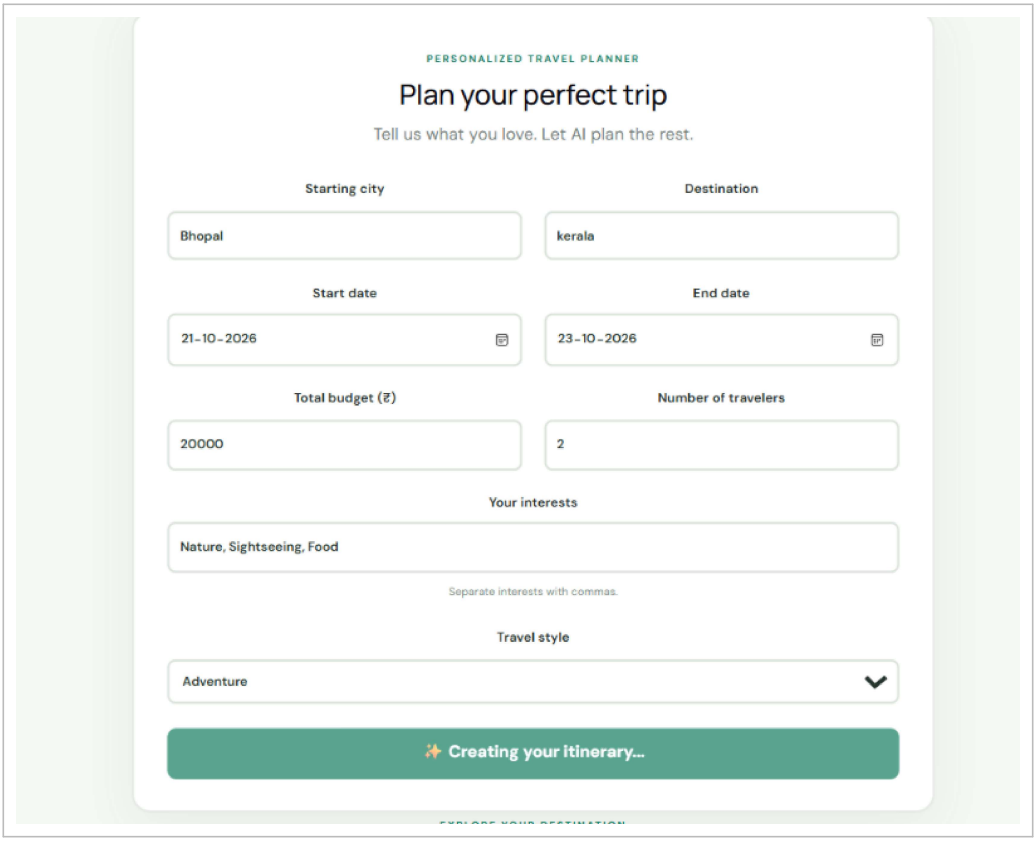


Figure 2. Filled trip form for the Bhopal-to-Kerala example.

Form fields

Field	Example	Use in the itinerary
Starting city / Destination	Bhopal / Kerala	Sets the route and the place context
Start and end date	21-10-2026 to 23-10-2026	Determines the number of days to plan
Total budget	20,000 INR	Guides estimated costs and activity choices
Number of travellers	2	Scales cost estimates for the group
Interests	Nature, Sightseeing, Food	Shapes the type of activities
Travel style	Adventure	Sets the pace and tone of the plan

7.2 Trip form and Saved Trips overview

A second supplied screenshot shows the trip input form and a Saved Trips card for a previously stored Pachmarhi trip. This demonstrates the intended information hierarchy: trip form first, followed by the saved-travel collection. Each saved card shows the travel style, destination, dates, traveller count and budget, with a View Itinerary button.

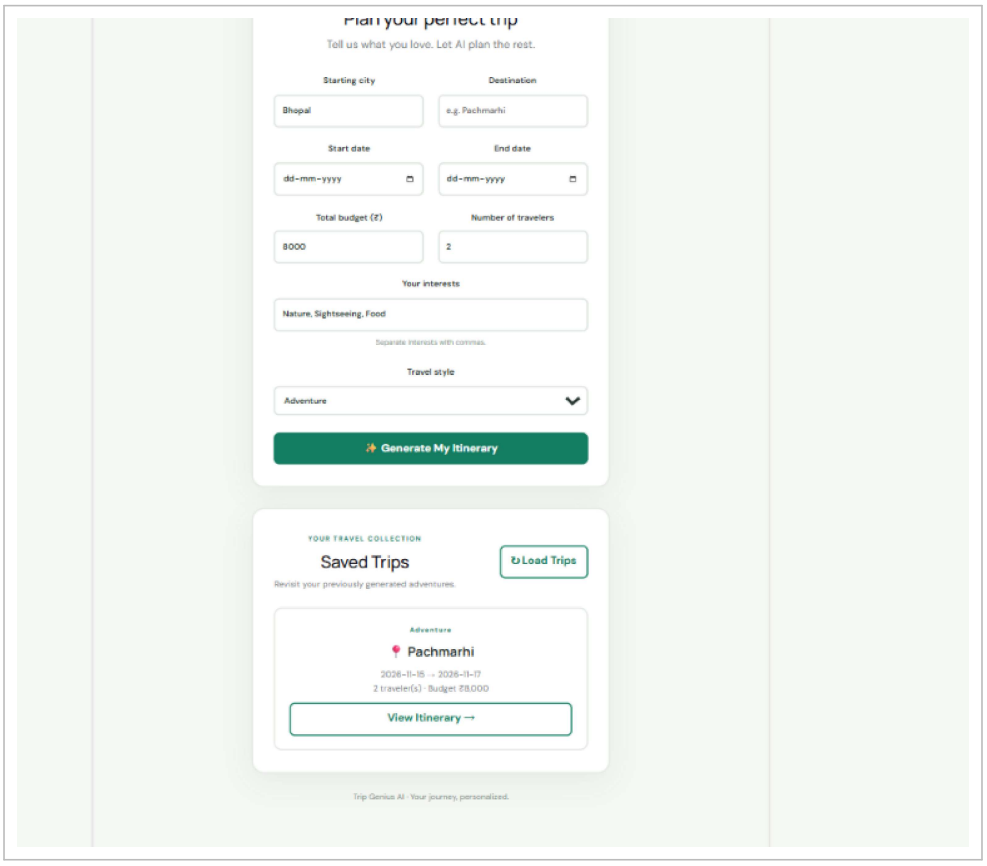


Figure 3. Trip Planner form and Saved Trips overview.

7.3 Generated itinerary view

The generated itinerary should present the destination summary, travel dates, day-wise activity plan, estimated cost and travel tips. Activities are grouped by day and by time of day so that the plan can be read quickly. A final evidence screenshot of the successfully generated Kerala itinerary should be inserted here if the original screenshot is available at full quality.

7.4 Hotels & Restaurants

The Hotels & Restaurants section provides a route from planning to external place discovery. Food preference terms are incorporated into the restaurant search query. Search results should be reviewed by the traveller, especially for dietary restrictions.

7.5 Saved Trips and print output

Saved Trips supports revisiting stored plans. The print/save action relies on the browser print dialogue. For final submission, include a successful saved-trip view and print preview captured after confirming that the deployed backend and database are working.

8. Testing and Validation Plan

The table lists the checks to be run against the deployed application. Each row states the behaviour that counts as a pass.

Test	Expected behaviour	Status
Backend health	/health responds successfully	Pending verification
Frontend loading	Home page renders without console/build errors	Pending verification
Itinerary generation	Valid request returns structured days and activities	Pending verification
Saved trips	Newly saved trip appears after reload	Pending verification
Retrieve trip	Selected saved trip opens correctly	Pending verification
Delete trip	Selected trip and related itinerary are removed	Pending verification
Map searches	Hotel/restaurant URL opens with requested query	Pending verification
Print/save	Browser print dialogue opens	Pending verification

These are planned checks, not claimed results. Execute them against the deployed app and replace each status with the observed outcome. No latency, throughput or success-rate values should be reported unless measured.

Suggested evidence

- A screenshot of the /health response in the browser.
- A screenshot of the generated itinerary and of the Saved Trips list after a page reload.
- A print preview showing the saved itinerary.
- A short note of any failed test, its cause and the fix applied.

9. Limitations, Security and Future Scope

Current limitations

- AI-generated activity suggestions and costs are estimates, not guaranteed quotations.
- Google Maps links do not mean the backend verifies individual businesses or live availability.
- A generated itinerary should be reviewed for travel time, local conditions and suitability.
- Availability of Groq and database services depends on configuration, quotas and connectivity.

Security considerations

- Keep API keys and database credentials on the server side.
- Review Supabase policies and ensure access to stored trips is appropriate.
- Restrict CORS to intended frontend origins for production.
- Avoid exposing private tokens or environment variables in screenshots and logs.

Future enhancements

- Automated unit, integration and end-to-end tests.
- User authentication and user-specific trip ownership.
- Approved live place-data and weather integrations.

- Authorized booking integrations for transport and accommodation.
- More transparent price estimation and currency handling.
- Accessibility, multilingual support, monitoring, rate limits and caching.

10. Deployment and References

Resource	Address
GitHub repository	https://github.com/Arpit0027-lab/trip-genius-ai
Frontend	https://trip-genius-ai-nine.vercel.app
Backend	https://trip-genius-ai.onrender.com
Health endpoint	https://trip-genius-ai.onrender.com/health

Implementation references: uploaded project source archive; frontend source under frontend/src; backend routes in backend/main.py; AI request and response processing in backend/ai_service.py; request/response models in backend/schemas.py; and database client configuration in backend/supabase_client.py.

Conclusion

Trip Genius AI brings together preference-based itinerary generation, persistence of saved trips, external hotel/restaurant searches and browser-based itinerary printing. The project separates the frontend, API, AI service and database responsibilities, which keeps the design easy to explain, test and extend.

Before final submission, the testing checklist should be completed with observed results and the final demonstration video URL should be added after recording. With authentication, live place data and automated tests added in future work, the application can grow from a planning assistant into a more complete travel-support tool.

Pre-submission checklist

- All eight tests in Section 8 executed and their status columns updated with observed outcomes.
- A screenshot of the generated Kerala itinerary inserted in Section 7.3.
- A saved-trip view and print preview captured from the deployed application.
- The demonstration video URL added to the Deployment and References table.
- No API keys, tokens or .env contents visible in any screenshot or log.